Semantic Tagging with GloVe and NumPy

Lab Report – Assignment 2

Name	Aman Banka
Registration No.	261100660030
Course Code	AME 5151
Course Name	Applied Linear Algebra Lab
Assignment	Assignment 2 (CO2) – Academic Year 2026–2027, Semester 1
Institution	Manipal School of Information Sciences, Manipal Academy of Higher Education (MAHE), Manipal
Faculty	Dr. Devi Prasad

1. Submitted files

File	Contents (submission requirement)
`manipal_linkedin_text.txt`	(1) Source text – 345 whitespace-separated tokens (≤ 400 words) of real LinkedIn post text about Manipal University (MAHE).
`SOURCES.md`	(2) Citation for every LinkedIn excerpt (also reproduced in Section 3 below).
`vector.py`, `semantic_tagging.py`	(3) Python source: tuple-based `Vector` class, and the complete pipeline (Parts A–H).
`test_semantic_tagging.py`	(4) Unit tests (all 7 required cases + 6 extra checks).
`program_output.txt`, `test_output.txt`	(5) Program output (token statistics, shapes, `np.allclose`, max difference, top-8 ranking) and test-run output.
`Lab_Report.pdf`	(6) This report: documentation and answers to the lab-report questions.

How to run: `python semantic_tagging.py` and `python -m pytest -v test_semantic_tagging.py`. The model is loaded with `gensim.downloader.load("glove-wiki-gigaword-50")`; if gensim is unavailable, the program reads the identical downloaded file `glove-wiki-gigaword-50.gz` (400 000 words × 50 dimensions) directly. Required: Python 3, NumPy (pytest for the tests).

2. Part A – Tag vocabulary

The 20 tags are:

`research, innovation, education, university, students, faculty, campus, engineering, medicine, technology, curriculum, collaboration, publication, laboratory, scholarship, mentorship, internship, entrepreneurship, alumni,` **`higher education`**

Justification. The tags cover the main activities of a research university in five groups: (i) *institution* – university, campus, higher education; (ii) *people* – students, faculty, alumni; (iii) *teaching and learning* – education, curriculum, scholarship, mentorship, internship; (iv) *research output* – research, laboratory, publication, innovation, collaboration,

entrepreneurship; (v) *disciplines* – engineering, medicine, technology. These match what MAHE posts about on LinkedIn (rankings, research, industry partnerships, health sciences, student programmes). The starter list was used with one change: *accreditation* was replaced by the multi-word tag *higher education*, because MAHE's own name and posts use that phrase and it shows the multi-word mechanism. Every tag word is in the GloVe vocabulary; `build_tag_matrix` checks this and raises `ValueError` otherwise.

Multi-word tags used: only one, **"higher education"**. Its tag vector is the mean v_{tag} = ½ (v_{higher} + $v_{education}$). All other 19 tags are single words with v_t = *model[t]*.

3. Part B – Input text and citations

The text file contains 17 verbatim excerpts from public LinkedIn posts about MAHE, separated by blank lines. It includes punctuation, names (Manu Gupta, Becton, Dickinson), numbers (2025, 11-50, 18th), hashtags (#SVASTH26, #StudentAchievement), repeated words (MAHE ×11, education ×6) and words outside the GloVe vocabulary. LinkedIn does not let automated tools open single post pages, so each excerpt was taken from the public "Activity" feed of the LinkedIn page shown below (accessed 3 November 2026).

#	Excerpt (opening words)	Post author / original poster	LinkedIn page
1	"Manipal Academy of Higher Education (MAHE), an Institution of Eminence … 3rd position in the National Institutional Ranking"	MAHE post, reposted by Punya A M	https://in.linkedin.com/in/punya1am
2	"In the Innovation ranking, MAHE proudly stands in the 11-50 band for 2025 …"	MAHE post, reposted by Punya A M	https://in.linkedin.com/in/punya1am
3	"As the Lead Support Organisation, Manipal Academy of Higher Education -MAHE was honored …"	Sonia Sharma (BMJ Research Forum South Asia 2026)	https://in.linkedin.com/in/soniash
4	"With the World Health Summit (WHS), we asked a bigger question …"	Manipal Academy of Higher Education – MAHE, Dubai (reposted by MAHE)	https://www.linkedin.com/in/manipal-academy-of-higher-education-mahe-79b42019b/
5	"Applications are now open for the Full-Time & Part-Time PhD Program 2025-26 …"	MAHE Directorate of Research (reposted by MAHE)	https://www.linkedin.com/in/manipal-academy-of-higher-education-mahe-79b42019b/
6	"MCES is thrilled to announce an open call for an Assistant Professor …"	MCES MAHE (Manipal Centre for European Studies)	https://in.linkedin.com/in/mces-mahe
7	"#MISHA #MAHE #StudentAchievement #ICWA …" (hashtags of a post)	Manipal Institute of Social Sciences Humanities and Arts (MISHA)	https://www.linkedin.com/in/manipal-institute-of-social-sciences-humanities-and-arts-misha-6ba0942b2/
8	"#UndergraduateResearch #ResearchConference …" (hashtags of a post)	Mohamed Shafeeq Karinkurayil	https://www.linkedin.com/in/mohamed-shafeeq-karinkurayil-24b097119/

9	"Glad to have been part of the Manipal Academy of Higher Education -Microsoft India collaboration event ..."	Bharath Kumar (shown on MAHE Bengaluru Alumni page)	https://www.linkedin.com/in/mahe-bengaluru-alumni-0797763b2/
10	"The Department of Student Affairs, MAHE Manipal, successfully conducted the Hostel Caretakers' ..."	Student Affairs MAHE	https://www.linkedin.com/in/student-affairs-mahe-764540191/
11	"MAHE is all set to embark on Aati da Tulu Parba 2026 ..."	Manipal Academy of Higher Education (shared by Student Affairs MAHE)	https://www.linkedin.com/in/student-affairs-mahe-764540191/
12	"The Clinical Immersion Program at Manipal welcomes its 18th cohort ..."	Corporate Relations, MAHE (shared by Dr Harish Kumar S)	https://www.linkedin.com/in/dr-harish-kumar-s-27253120b/
13	"Manipal Academy of Higher Education (MAHE) has signed a Collaboration Agreement with Google ..."	Corporate Relations, MAHE (shared by Dr Harish Kumar S)	https://www.linkedin.com/in/dr-harish-kumar-s-27253120b/
14	"Where technology meets healthcare, ideas meet impact ... #SVASTH26 ..."	Department of Healthcare and Hospital Management, PSPH, MAHE	https://in.linkedin.com/in/dhhm
15	"Dear MITians, Pleased to inform you that Mr. Manu Gupta ..."	Post shown on LinkedIn profile	https://hu.linkedin.com/in/daniel-janicsak
16	"The DoC MAHE- Bengaluru recently organized a One-week international Immersion ..."	Department of Commerce MAHE Blr (reposted by MAHE Bengaluru)	https://in.linkedin.com/company/mahe-bengaluru
17	"At MAHE- Bengaluru we conducted a bird count last Monday..."	Deepta Sateesh (reposted by MAHE Bengaluru)	https://in.linkedin.com/company/mahe-bengaluru

4. Part C – Preprocessing

Steps in the required order (function `preprocess`): (1) `text.lower()`; (2) every character that is not a letter, digit or whitespace (and `_`) is replaced by a space – so `#MAHE` → `mahe`, `deep-tech` → `deep tech`, and en/em dashes disappear; (3) split on whitespace; (4) remove stopwords; (5) remove tokens with length ≤ 2; (6) `build_text_matrix` splits the tokens into in-vocabulary and out-of-vocabulary (OOV) using `model.key_to_index`. Repeated in-vocabulary words are kept, so *mahe* gives 11 identical rows of W.

Stopwords. The 18-word starter set was used. I added 88 words that occur in LinkedIn text: pronouns and possessives (*i, me, my, we, us, our, ours, you, your, he, him, his, she, her, it, its, they, them, their*); demonstratives and determiners (*this, that, these, those, all, any, some, each, every*); auxiliaries and modals (*be, been, being, am, has, have, had, do, does, did, can, could, will, would, shall, should, may, might, must*); and other function words (*as, if, so, but, not, no, nor, than, then, too, very, into, onto, over, under, up, out,*

about, between, through, during, before, after, beyond, while, how, what, which, who, whom, where, when, why, there, here, also, just, now, own, per, via). No external stopword library is used.

Token statistic	Value
Tokens before preprocessing	345
Tokens after preprocessing (m)	232
In-vocabulary tokens (n)	217
Out-of-vocabulary tokens	15 (15 distinct)

Distinct OOV tokens: `digitalhealth`, `healthcareinnovation`, `hospitalmanagement`, `interdisciplinaryresearch`, `internationalrelations`, `mces`, `mitians`, `parba`, `psph`, `researchconference`, `researchexcellence`, `studentachievement`, `studentresearch`, `svasth26`, `undergraduateresearch`. Most are hashtags made of joined words; the rest are acronyms (mces, psph), a coined word (mitians) and a Tulu word (parba).

5. Parts D–G – Matrices, two implementations, verification

Part D. `build_tag_matrix` stacks the 20 tag vectors as rows: $T \in \mathbb{R}^{20\times 50}$. `build_text_matrix` stacks the 217 in-vocabulary token vectors in text order: $W \in \mathbb{R}^{217\times 50}$. Checkpoint output: `T.shape = (20, 50), W.shape = (217, 50)`. If $n = 0$, the program stops with an error message before computing similarities.

Part E. `similarity_matrix_vector_class` uses two nested Python loops over $i = 1..n$ and $j = 1..20$, and for each pair builds `Vector(W[i])` and `Vector(T[j])` and calls `cosine_similarity`, which uses only the pure-Python `dot` and `norm` methods (no NumPy). This gives 4 340 entries.

Part F. `similarity_matrix_numpy` computes the row norms, checks that none is zero, normalises the rows and computes $S = \hat{W}\,\hat{T}^T$ with one `@`: (217×50)(50×20) = 217×20. No library cosine function is used.

Part G. Both matrices are kept in float64 (the GloVe float32 values are converted to float64 when W and T are built), so the only differences come from rounding. Tolerance: `rtol = 1e-9, atol = 1e-12`. This is much stricter than NumPy's default and still far above float64 rounding error ($\approx 2.2\times10^{-16}$).

Check	Result
`np.allclose(S_vector, S_numpy, rtol=1e-9, atol=1e-12)`	**True**
`np.max(np.abs(S_vector - S_numpy))`	**5.551×10^{-16}** ($\approx$ 2.5 machine epsilon)

6. Part H – Max-pool ranking

`rank_tags` computes $r_j = max_i\ S_{ij}$ (column-wise maximum), stores the word at the arg-max row, and sorts the tags by score in descending order.

Rank	Tag	Max-pool score	Best-matching text word	Match type
1	research	1.0000	research	identical word
2	innovation	1.0000	innovation	identical word
3	university	1.0000	university	identical word
4	education	1.0000	education	identical word
5	technology	1.0000	technology	identical word
6	collaboration	1.0000	collaboration	identical word
7	higher education	0.8871	education	different word (a component of the tag)
8	students	0.8824	student	different word (singular form)

The six tags with score exactly 1 tie at the top because the tag word itself appears in the text. Ranks 9–20 (from the full program output): faculty 0.826 (university), curriculum 0.809 (educational), entrepreneurship 0.796 (innovation), laboratory 0.792 (research), medicine 0.785 (studies), engineering 0.778 (research), campus 0.770 (university), alumni 0.749 (alumnus), scholarship 0.719 (university), publication 0.606 (conducted), internship 0.576 (program), mentorship 0.555 (educational).

7. Unit tests

All 13 tests pass (`test_output.txt`). The tests load the real GloVe model and use a short test sentence that contains punctuation, a hashtag, a number, a repeated word, stopwords and two invented words that are not in the vocabulary.

Test	Required case checked
test_01_tag_matrix_shape	1. `T.shape == (20, 50)`
test_01b_tag_rows	Single-word row = `model[tag]`; “higher education” row = mean of the two word vectors
test_02_text_matrix_shape	2. `W.shape == (n, 50)`; repeated word kept twice; row i = vector of token i
test_03_similarity_shape	3. `S.shape == (n, 20)` for both implementations
test_04_single_pair_agrees	4. (university, campus): `Vector` cosine = NumPy entry; identical pair gives 1
test_05_allclose	5. `np.allclose(S_vector, S_numpy)` is True
test_06_oov_excluded_and_reported	6. OOV tokens (qwzxtech, blorptastic, researchexcellence) are not in W and are listed as OOV
test_07_oov_tag_raises	7. An OOV single-word tag and an OOV component of a multi-word tag both raise `ValueError`
test_08 … test_11, test_vector_ops	Extra: preprocessing order, $n = 0$ handling, max-pool correctness, zero-row rejection, `Vector` operations

8. Answers to the lab-report questions

Q1. Dimensions, with n in-vocabulary words (here n = 217).

(a) One word vector: $w \in \mathbb{R}^{50}$. As a row of W it is 1×50 (as a column vector, 50×1).
(b) W: n×50 = 217×50.
(c) T: 20×50.
(d) T^T: 50×20.
(e) $S = \hat{W}\,\hat{T}^T$: (n×50)(50×20) = n×20 = 217×20.

Q2. Why is $\hat{\mathbf{W}}_{\mathbf{i},:}\,\hat{\mathbf{T}}_{\mathbf{j},:}^{\mathbf{T}}$ the cosine similarity of the original vectors?

Write $w = W_{i,:}$ and $t = T_{j,:}$. Both are nonzero (the code checks this), so $\hat{W}_{i,:} = w/\|w\|_2$ and $\hat{T}_{j,:} = t/\|t\|_2$. The product of a 1×50 row with a 50×1 column is the dot product. Because the dot product is linear in each argument, the scalars $1/\|w\|$ and $1/\|t\|$ can be taken out:

$$\hat{W}_{i,:}\,\hat{T}_{j,:}^{\,T} = (w/\|w\|)\cdot(t/\|t\|) = (w\cdot t)\,/\,(\|w\|\,\|t\|) = \cos(w, t).$$

So after normalisation, the dot product of two unit vectors is the cosine of the angle between them. Scaling a vector does not change its direction, so it does not change the cosine. Normalising first just moves the division by the norms out of every pairwise calculation.

Q3. Why does $\hat{\mathbf{W}}\,\hat{\mathbf{T}}^{\mathbf{T}}$ compute all 20n cosine similarities in one multiplication?

By the definition of matrix multiplication, $(AB)_{ij}$ = (row i of A) · (column j of B). Column j of $\hat{T}^T$ is row j of $\hat{T}$, the unit tag vector. Therefore

$$(\hat{W}\,\hat{T}^T)_{ij} = \Sigma_{k=1..50}\,\hat{W}_{ik}\,\hat{T}_{jk} = \hat{W}_{i,:}\cdot\hat{T}_{j,:} = \cos(W_{i,:}, T_{j,:}),\ \text{for every } i = 1,\ldots,n \text{ and } j = 1,\ldots,20.$$

The (n×50)(50×20) product has n·20 entries, and each one is exactly one word–tag dot product (Q2). So one product gives all 20n cosines (4 340 here), using 50·20n multiply-adds inside NumPy's compiled matrix routine instead of a Python double loop.

Q4. One word has very high similarity to “medicine” and every other word has low similarity. How does max pooling handle this?

The score is $r_{medicine} = max_i\ S_{i,medicine}$, so it equals the value of that one word. All the low values in the column are ignored: they do not lower the score, and neither does the number of words in the text. The tag therefore gets a high score and can rank near the top because of a single word. Max pooling is an “any strong evidence” rule. An average (mean pooling) would behave differently: one high value among many low ones would be diluted. In our text the “medicine” column has a maximum of 0.785 (word *studies*) but a column mean of only 0.314. This shows how much max pooling depends on one entry.

Q5. Does a high max-pool score mean the tag describes the dominant topic?

No. The maximum depends on one entry of the column, so it does not measure how often or how widely a topic appears. One strongly related word is enough. In our text *university* appears once and *collaboration* twice, but they score 1.0, exactly the same as *education*, which appears 6 times. Rankings and the institution itself (*mahe* ×11, *manipal* ×8) are the most frequent themes, but no tag captures them directly. A high max-pool score shows strong local evidence that the topic is present, not that it dominates. To measure dominance you would need a statistic that uses the whole

column, such as the mean, a count of S_{ij} above a threshold, or a sum over the top k words.

Q6. Identical vs different matches in my top eight, and what this shows about cosine similarity over embeddings.

Identical word (score exactly 1): research, innovation, university, education, technology, collaboration (ranks 1–6).
Different word: higher education ← *education* (0.887) and students ← *student* (0.882).

The identical matches are trivial: $\cos(v, v) = 1$ for any vector, so these scores only show that the text contains the same string. They say nothing about meaning, and they take the top places ahead of every real semantic match. The two different-word matches are only partly different. *education* is a component of the multi-word tag, so the tag vector is the average of *education* and *higher*, and the cosine is high but below 1. *student* vs *students* is only a singular/plural difference. GloVe gives each surface form its own vector, so the model has to learn their similarity from context (0.88) instead of from spelling.

The lower ranks show what the embeddings actually capture: alumni ← *alumnus*, curriculum ← *educational*, faculty ← *university*, but also medicine ← *studies* (0.785, higher than *clinical* at 0.72) and engineering ← *research* (higher than *engineers*). GloVe vectors are built from word co-occurrence statistics, so cosine similarity measures *distributional relatedness*: words used in similar contexts or about the same broad topic score high. Cosine similarity is **not** synonymy and not a judgment of meaning in this text. It has no word sense and no context (each word has one fixed vector), no negation, no phrase meaning beyond simple averaging, and it rewards frequent generic words (*studies, research, university, program*). It finds “related to the same topic”, not “means the same thing”.

Q7. Can the ranking algorithm introduce a new tag for a concept not in the 20 tags?

No. The output space is fixed by T: $S = \hat{W}\,\hat{T}^{T}$ has exactly 20 columns, one per row of T. Max pooling only computes a score for each existing column and sorts them, so it can only reorder the given 20 tags. A concept with no tag (for example rankings, culture or healthcare management in our text) can show up only indirectly: low maximum scores for every tag, or its words appearing as the best match for an unrelated tag. Adding a tag needs a new row in T, chosen by a person (or by a different algorithm, such as clustering the text vectors). Max-pool ranking is a closed-vocabulary classifier, not a method for discovering topics.

Q8. Why are OOV tokens excluded from W?

Each row of W must be the model's embedding of a word, and an OOV token has no embedding. Any row we invented would make the result wrong or undefined. A zero row has norm 0, so its cosine (division by $\|w\|$) is undefined. A random or average vector would produce similarities with no meaning that could even become a row's maximum and distort the ranking. Leaving these tokens out keeps every row of W a real model vector and every entry of S a well-defined cosine. The tokens are still counted and listed so that no information is hidden. In our text they are mostly joined hashtags (*studentachievement*, *researchexcellence*) and acronyms (*mces*, *psph*). This also shows a limitation: the preprocessing does not split hashtags into their component words.

Q9. Mathematical relationship between the pairwise Vector.cosine_similarity computation and the NumPy product.

Both compute the same number for every (i, j):

$$S_{ij} = \Sigma_k \, W_{ik} T_{jk} \, / \, (\sqrt{(\Sigma_k \, W_{ik}^2)} \cdot \sqrt{(\Sigma_k \, T_{jk}^2)} \,).$$

The `Vector` version evaluates this formula entry by entry: $20n$ separate dot products, recomputing both norms for every pair. The NumPy version factors the same formula. Let $D_W = \mathrm{diag}(\|W_{1,:}\|, \ldots, \|W_{n,:}\|)$ and $D_T = \mathrm{diag}(\|T_{1,:}\|, \ldots, \|T_{20,:}\|)$. Then $\hat{W} = D_W^{-1} W$, $\hat{T} = D_T^{-1} T$, and

$$S = \hat{W} \, \hat{T}^T = D_W^{-1} \, (W \, T^T) \, D_T^{-1}.$$

Here $W \, T^T$ is the matrix of all raw dot products (the numerators). Left-multiplying by D_W^{-1} divides row i by $\|W_{i,:}\|$, and right-multiplying by D_T^{-1} divides column j by $\|T_{j,:}\|$. Entry by entry this is the formula above. Each norm is computed once ($n + 20$ norms instead of $2 \cdot 20n$), and the double loop becomes one matrix product. The two results are mathematically identical. They differ only in floating-point rounding, because the sums are evaluated in a different order and with different operations. That is why we compare them with `np.allclose` and not with exact equality. The observed maximum difference, 5.6×10^{-16}, is on the order of float64 machine epsilon.

Appendix A – Program output (`python semantic_tagging.py`)

```
Name             : Aman Banka
Registration No. : 261100660030
Course           : AME 5151 - Applied Linear Algebra Lab
Assignment       : Assignment 2 - Semantic Tagging with GloVe and NumPy
Institution      : Manipal School of Information Sciences, MAHE, Manipal
Faculty          : Dr. Devi Prasad

================================================================================
Semantic Tagging with GloVe (glove-wiki-gigaword-50) and NumPy
================================================================================
Model loaded: 400000 words, dimension 50

[Part A] 20 tags: research, innovation, education, university, students, faculty, campus,
engineering, medicine, technology, curriculum, collaboration, publication, laboratory,
scholarship, mentorship, internship, entrepreneurship, alumni, higher education
         multi-word tags (mean of component vectors): ['higher education']

[Part B] Input file: manipal_linkedin_text.txt

[Part C] Token statistics
  Tokens before preprocessing        : 345
  Tokens after preprocessing (m)     : 232
  In-vocabulary tokens (n)           : 217
  Out-of-vocabulary tokens           : 15
  Distinct out-of-vocabulary tokens  : 15
    ['digitalhealth', 'healthcareinnovation', 'hospitalmanagement',
'interdisciplinaryresearch', 'internationalrelations', 'mces', 'mitians', 'parba', 'psph',
'researchconference', 'researchexcellence', 'studentachievement', 'studentresearch',
'svasth26', 'undergraduateresearch']
  Stopwords added to the starter set : 88

[Part D] Checkpoint
  T.shape = (20, 50)
  W.shape = (217, 50)

[Part E] S_vector.shape = (217, 20)  (Vector class, Python loops)
[Part F] S_numpy.shape  = (217, 20)  (W_hat @ T_hat.T)

[Part G] Verification
  np.allclose(S_vector, S_numpy, rtol=1e-09, atol=1e-12) -> True
  np.max(np.abs(S_vector - S_numpy)) = 5.551e-16

[Part H] Top 8 tags by max-pool score  r_j = max_i S_ij

Rank  Tag                Score    Best-matching text word   Match type
--------------------------------------------------------------------------------
1     research           1.0000   research                 identical word
2     innovation         1.0000   innovation               identical word
3     university         1.0000   university               identical word
4     education          1.0000   education                identical word
5     technology         1.0000   technology               identical word
6     collaboration      1.0000   collaboration            identical word
7     higher education   0.8871   education                different word
8     students           0.8824   student                  different word

  Top-8 tags matched by an IDENTICAL word : ['research', 'innovation', 'university',
'education', 'technology', 'collaboration']
  Top-8 tags matched by a DIFFERENT word  : ['higher education <- education', 'students <-
student']

  Full ranking of all 20 tags (for reference):
```

```
Rank  Tag                Score    Best-matching text word   Match type
------------------------------------------------------------------------------
1     research           1.0000   research                  identical word
2     innovation         1.0000   innovation                identical word
3     university         1.0000   university                identical word
4     education          1.0000   education                 identical word
5     technology         1.0000   technology                identical word
6     collaboration      1.0000   collaboration             identical word
7     higher education   0.8871   education                 different word
8     students           0.8824   student                   different word
9     faculty            0.8258   university                different word
10    curriculum         0.8087   educational               different word
11    entrepreneurship   0.7957   innovation                different word
12    laboratory         0.7919   research                  different word
13    medicine           0.7853   studies                   different word
14    engineering        0.7780   research                  different word
15    campus             0.7697   university                different word
16    alumni             0.7485   alumnus                   different word
17    scholarship        0.7189   university                different word
18    publication        0.6058   conducted                 different word
19    internship         0.5761   program                   different word
20    mentorship         0.5545   educational               different word
```

Appendix B – Unit-test output (`python -m pytest -v test_semantic_tagging.py`)

```
Name             : Aman Banka
Registration No. : 261100660030
Course           : AME 5151 - Applied Linear Algebra Lab
Assignment       : Assignment 2 - Semantic Tagging with GloVe and NumPy
Institution      : Manipal School of Information Sciences, MAHE, Manipal
Faculty          : Dr. Devi Prasad

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /home/claude/submission
plugins: platformdirs-4.13.3, anyio-4.15.1
collecting ... collected 13 items

test_semantic_tagging.py::TestSemanticTagging::test_01_tag_matrix_shape PASSED [  7%]
test_semantic_tagging.py::TestSemanticTagging::test_01b_tag_rows PASSED  [ 15%]
test_semantic_tagging.py::TestSemanticTagging::test_02_text_matrix_shape PASSED [ 23%]
test_semantic_tagging.py::TestSemanticTagging::test_03_similarity_shape PASSED [ 30%]
test_semantic_tagging.py::TestSemanticTagging::test_04_single_pair_agrees PASSED [ 38%]
test_semantic_tagging.py::TestSemanticTagging::test_05_allclose PASSED   [ 46%]
test_semantic_tagging.py::TestSemanticTagging::test_06_oov_excluded_and_reported PASSED [
53%]
test_semantic_tagging.py::TestSemanticTagging::test_07_oov_tag_raises PASSED [ 61%]
test_semantic_tagging.py::TestSemanticTagging::test_08_preprocessing PASSED [ 69%]
test_semantic_tagging.py::TestSemanticTagging::test_09_empty_text_matrix PASSED [ 76%]
test_semantic_tagging.py::TestSemanticTagging::test_10_rank_tags PASSED  [ 84%]
test_semantic_tagging.py::TestSemanticTagging::test_11_zero_row_rejected PASSED [ 92%]
test_semantic_tagging.py::TestVectorClass::test_vector_ops PASSED        [100%]

============================== 13 passed in 3.80s ==============================
```

Appendix C – Source code

vector.py

```
"""
Name             : Aman Banka
Registration No. : 261100660030
Course           : AME 5151 - Applied Linear Algebra Lab
Assignment       : Assignment 2 - Semantic Tagging with GloVe and NumPy
Institution      : Manipal School of Information Sciences, MAHE, Manipal
Faculty          : Dr. Devi Prasad
"""

"""
vector.py -- the tuple-based Vector class built earlier in the course.

Storage is an immutable tuple of Python floats. All arithmetic is done in
pure Python (no NumPy), so this class acts as an independent reference
implementation for checking the vectorised NumPy code.
"""

import math


class Vector:
    """An immutable vector in R^d stored as a tuple of floats."""

    def __init__(self, components):
        self._data = tuple(float(c) for c in components)
        if len(self._data) == 0:
            raise ValueError("A Vector must have at least one component.")

    # ----- basic container behaviour -------------------------------------
    @property
    def components(self):
        return self._data

    def __len__(self):
        return len(self._data)

    def __getitem__(self, i):
        return self._data[i]

    def __iter__(self):
        return iter(self._data)

    def __repr__(self):
        return f"Vector({self._data})"

    def __eq__(self, other):
        return isinstance(other, Vector) and self._data == other._data

    def _check_dim(self, other):
        if not isinstance(other, Vector):
            raise TypeError("Operand must be a Vector.")
        if len(self) != len(other):
            raise ValueError(
                f"Dimension mismatch: {len(self)} vs {len(other)}"
            )

    # ----- vector operations ---------------------------------------------
    def add(self, other):
        """u + v (component-wise)."""
        self._check_dim(other)
```

```
        return Vector(a + b for a, b in zip(self._data, other._data))

    def scale(self, c):
        """c * u for a scalar c."""
        return Vector(c * a for a in self._data)

    def dot(self, other):
        """u . v = sum_k u_k v_k."""
        self._check_dim(other)
        return sum(a * b for a, b in zip(self._data, other._data))

    def norm(self):
        """Euclidean norm ||u||_2 = sqrt(u . u)."""
        return math.sqrt(self.dot(self))

    def cosine_similarity(self, other):
        """cos(u, v) = (u . v) / (||u||_2 ||v||_2) for nonzero u, v."""
        self._check_dim(other)
        nu, nv = self.norm(), other.norm()
        if nu == 0.0 or nv == 0.0:
            raise ValueError("Cosine similarity is undefined for a zero vector.")
        return self.dot(other) / (nu * nv)

    # operator sugar
    __add__ = add

    def __mul__(self, c):
        return self.scale(c)

    __rmul__ = __mul__
```

semantic_tagging.py

```
"""
Name             : Aman Banka
Registration No. : 261100660030
Course           : AME 5151 - Applied Linear Algebra Lab
Assignment       : Assignment 2 - Semantic Tagging with GloVe and NumPy
Institution      : Manipal School of Information Sciences, MAHE, Manipal
Faculty          : Dr. Devi Prasad
"""

"""
AME 5151 - Applied Linear Algebra Lab, Assignment 2
Semantic Tagging with GloVe and NumPy

Pipeline
    Part A  20-tag vocabulary                          -> TAGS
    Part B  input text (<= 400 words, LinkedIn)        -> manipal_linkedin_text.txt
    Part C  preprocessing                              -> preprocess()
    Part D  tag matrix T (20x50), text matrix W (nx50)-> build_tag_matrix(),
build_text_matrix()
    Part E  S via the Vector class (Python loops)      -> similarity_matrix_vector_class()
    Part F  S via NumPy  S = W_hat @ T_hat.T           -> similarity_matrix_numpy()
    Part G  verification with np.allclose              -> verify()
    Part H  max-pool tag ranking                       -> rank_tags()

Run:   python semantic_tagging.py [path/to/text.txt]
"""

import os
import re
import sys

import numpy as np
```

```
from vector import Vector

MODEL_NAME = "glove-wiki-gigaword-50"
EMBED_DIM = 50
HERE = os.path.dirname(os.path.abspath(__file__))
DEFAULT_TEXT = os.path.join(HERE, "manipal_linkedin_text.txt")

# ---------------------------------------------------------------------------
# Part A -- tag vocabulary (exactly 20 tags; one multi-word tag)
# ---------------------------------------------------------------------------
TAGS = [
    "research",
    "innovation",
    "education",
    "university",
    "students",
    "faculty",
    "campus",
    "engineering",
    "medicine",
    "technology",
    "curriculum",
    "collaboration",
    "publication",
    "laboratory",
    "scholarship",
    "mentorship",
    "internship",
    "entrepreneurship",
    "alumni",
    "higher education",   # multi-word tag: mean of v_higher and v_education
]

# ---------------------------------------------------------------------------
# Part C -- stopword set (defined by hand; no external stopword library)
# ---------------------------------------------------------------------------
BASE_STOPWORDS = {
    "the", "a", "an", "and", "or", "of", "to",
    "in", "on", "for", "is", "are", "was", "were",
    "with", "at", "by", "from",
}
ADDED_STOPWORDS = {
    # pronouns / possessives
    "i", "me", "my", "we", "us", "our", "ours", "you", "your", "he", "him",
    "his", "she", "her", "it", "its", "they", "them", "their",
    # demonstratives / determiners
    "this", "that", "these", "those", "all", "any", "some", "each", "every",
    # auxiliaries / modals
    "be", "been", "being", "am", "has", "have", "had", "do", "does", "did",
    "can", "could", "will", "would", "shall", "should", "may", "might", "must",
    # other function words, conjunctions, prepositions
    "as", "if", "so", "but", "not", "no", "nor", "than", "then", "too", "very",
    "into", "onto", "over", "under", "up", "out", "about", "between", "through",
    "during", "before", "after", "beyond", "while", "how", "what", "which",
    "who", "whom", "where", "when", "why", "there", "here", "also", "just",
    "now", "own", "per", "via",
}
STOPWORDS = BASE_STOPWORDS | ADDED_STOPWORDS

# Everything that is not a letter, digit or whitespace counts as punctuation
# (this includes '#', '-', '&', en/em dashes, quotes and '_').
_PUNCT_RE = re.compile(r"[^\w\s]|_")


# ---------------------------------------------------------------------------
```

```
# Model loading
# ---------------------------------------------------------------------------
class _PlainKeyedVectors:
    """Minimal stand-in for gensim's KeyedVectors (key_to_index + model[word]).

    Used only when gensim is not installed; it reads exactly the same file
    that gensim.downloader fetches for glove-wiki-gigaword-50.
    """

    def __init__(self, words, vectors):
        self.index_to_key = words
        self.key_to_index = {w: i for i, w in enumerate(words)}
        self.vectors = vectors
        self.vector_size = vectors.shape[1]

    def __contains__(self, word):
        return word in self.key_to_index

    def __getitem__(self, word):
        return self.vectors[self.key_to_index[word]]


def _load_word2vec_text(path):
    import gzip
    opener = gzip.open if path.endswith(".gz") else open
    with opener(path, "rt", encoding="utf-8") as f:
        n, d = map(int, f.readline().split())
        words, vecs = [], np.empty((n, d), dtype=np.float32)
        for i, line in enumerate(f):
            parts = line.rstrip().split(" ")
            words.append(parts[0])
            vecs[i] = np.asarray(parts[1:], dtype=np.float32)
    return _PlainKeyedVectors(words, vecs)


_MODEL_CACHE = {}


def load_model():
    """Load glove-wiki-gigaword-50.

    1. gensim.downloader.load(MODEL_NAME)  (standard route)
    2. fallback: the downloaded file glove-wiki-gigaword-50.gz on disk
       (read with gensim if available, else with a small plain loader).
    """
    if "model" in _MODEL_CACHE:
        return _MODEL_CACHE["model"]
    model = None
    try:
        import gensim.downloader as api
        model = api.load(MODEL_NAME)
    except Exception:
        candidates = [
            os.environ.get("GLOVE_PATH", ""),
            os.path.join(HERE, f"{MODEL_NAME}.gz"),
            os.path.join(HERE, "..", "data", f"{MODEL_NAME}.gz"),
            os.path.expanduser(f"~/gensim-data/{MODEL_NAME}/{MODEL_NAME}.gz"),
        ]
        path = next((p for p in candidates if p and os.path.exists(p)), None)
        if path is None:
            raise RuntimeError(
                "Could not load glove-wiki-gigaword-50. Install gensim "
                "(pip install gensim) or set GLOVE_PATH to the .gz file."
            )
        try:
            from gensim.models import KeyedVectors
```

```
            model = KeyedVectors.load_word2vec_format(path, binary=False)
        except ImportError:
            model = _load_word2vec_text(path)
    _MODEL_CACHE["model"] = model
    return model


# ---------------------------------------------------------------------------
# Part C -- preprocessing
# ---------------------------------------------------------------------------
def preprocess(text, stopwords=STOPWORDS):
    """Steps 1-5 of Part C, in the required order.

    Returns (raw_token_count, tokens) where tokens = x_1, ..., x_m.
    Step 6 (vocabulary check) is done in build_text_matrix().
    """
    raw_token_count = len(text.split())          # tokens before preprocessing
    text = text.lower()                          # 1. lowercase
    text = _PUNCT_RE.sub(" ", text)              # 2. remove punctuation
    tokens = text.split()                        # 3. tokenise on whitespace
    tokens = [t for t in tokens if t not in stopwords]   # 4. stopwords
    tokens = [t for t in tokens if len(t) > 2]           # 5. length filter
    return raw_token_count, tokens


# ---------------------------------------------------------------------------
# Part D -- tag matrix and text matrix
# ---------------------------------------------------------------------------
def build_tag_matrix(model, tags):
    """
    Construct the tag matrix.

    Returns:
        tag_names: list[str] containing exactly 20 tag names.
        T:         NumPy array with shape (20, 50).

    Single-word tag  -> row = model[tag].
    Multi-word tag   -> row = arithmetic mean of the component word vectors.
    Every word used in a tag must occur in the model vocabulary; otherwise a
    ValueError is raised (the tag is never silently dropped).
    """
    tag_names = list(tags)
    if len(tag_names) != 20:
        raise ValueError(f"Exactly 20 tags are required, got {len(tag_names)}.")
    if len(set(tag_names)) != 20:
        raise ValueError("Tags must be distinct.")

    rows = []
    for tag in tag_names:
        words = tag.lower().split()
        if not words:
            raise ValueError("Empty tag.")
        for w in words:
            if w not in model.key_to_index:
                raise ValueError(f"Tag not in vocabulary: {tag!r} (word {w!r})")
        vecs = np.array([model[w] for w in words], dtype=np.float64)
        rows.append(vecs.mean(axis=0))           # k = 1 gives model[tag] itself
    T = np.vstack(rows)
    assert T.shape == (20, EMBED_DIM)
    return tag_names, T


def build_text_matrix(model, tokens):
    """
    Construct the text matrix.
```

```
    Parameters:
        tokens: preprocessed tokens (after lowercasing, punctuation removal,
                stopword removal and the length filter).

    Returns:
        in_vocab_tokens: tokens in the model vocabulary, original order and
                         repetitions preserved.
        oov_tokens:      tokens not in the model vocabulary (in order).
        W:               NumPy array (n, 50); row i = embedding of
                         in_vocab_tokens[i].
    """
    in_vocab_tokens = [t for t in tokens if t in model.key_to_index]
    oov_tokens = [t for t in tokens if t not in model.key_to_index]
    if in_vocab_tokens:
        W = np.array([model[t] for t in in_vocab_tokens], dtype=np.float64)
    else:
        W = np.empty((0, EMBED_DIM), dtype=np.float64)
    return in_vocab_tokens, oov_tokens, W


# ---------------------------------------------------------------------------
# Part E -- similarity matrix with the Vector class (reference computation)
# ---------------------------------------------------------------------------
def similarity_matrix_vector_class(W, T):
    """
    Compute the complete cosine-similarity matrix using the Vector class.

    Every entry is computed by constructing two Vector instances and calling
    cosine_similarity -- no NumPy dot/norm/matmul is used for the values.

    Returns S as a nested Python list of shape (n_words, 20), where
        S[i][j] = Vector(W[i]).cosine_similarity(Vector(T[j])).
    """
    n, m = len(W), len(T)
    S = []
    for i in range(n):
        row = []
        for j in range(m):
            w_vec = Vector(W[i].tolist())     # plain Python floats
            t_vec = Vector(T[j].tolist())
            row.append(w_vec.cosine_similarity(t_vec))
        S.append(row)
    return S


# ---------------------------------------------------------------------------
# Part F -- vectorised similarity matrix with NumPy
# ---------------------------------------------------------------------------
def similarity_matrix_numpy(W, T):
    """
    Compute all text-word/tag cosine similarities using NumPy row
    normalisation and one matrix multiplication.

    Returns S with shape (n_words, 20).
    """
    W = np.asarray(W, dtype=np.float64)
    T = np.asarray(T, dtype=np.float64)
    W_norm = np.linalg.norm(W, axis=1, keepdims=True)    # (n, 1)
    T_norm = np.linalg.norm(T, axis=1, keepdims=True)    # (20, 1)
    if np.any(W_norm == 0):
        raise ValueError("W contains a zero row; cosine similarity undefined.")
    if np.any(T_norm == 0):
        raise ValueError("T contains a zero row; cosine similarity undefined.")
    W_hat = W / W_norm                                   # (n, 50), unit rows
    T_hat = T / T_norm                                   # (20, 50), unit rows
    S = W_hat @ T_hat.T                                  # (n,50)(50,20) = (n,20)
```

```
    return S


# ---------------------------------------------------------------------------
# Part G -- verification
# ---------------------------------------------------------------------------
RTOL = 1e-9
ATOL = 1e-12


def verify(S_vector, S_numpy, rtol=RTOL, atol=ATOL):
    """Return (allclose_flag, max_abs_difference)."""
    S_vector = np.array(S_vector, dtype=np.float64)
    ok = bool(np.allclose(S_vector, S_numpy, rtol=rtol, atol=atol))
    max_diff = float(np.max(np.abs(S_vector - S_numpy)))
    return ok, max_diff


# ---------------------------------------------------------------------------
# Part H -- max-pool tag ranking
# ---------------------------------------------------------------------------
def rank_tags(tag_names, S, in_vocab_tokens):
    """
    For each tag j:  score[j] = max_i S[i, j].
    Return the tags sorted in descending order of score, each as a dict with
    the tag, its score, the text token that produced the maximum, and whether
    that token is identical to the tag (exact match -> similarity 1).
    """
    S = np.asarray(S)
    best_rows = np.argmax(S, axis=0)                    # arg max over words
    scores = S[best_rows, np.arange(S.shape[1])]        # max over words
    results = []
    for j, tag in enumerate(tag_names):
        word = in_vocab_tokens[best_rows[j]]
        results.append({
            "tag": tag,
            "score": float(scores[j]),
            "word": word,
            "identical": word == tag,
        })
    results.sort(key=lambda r: r["score"], reverse=True)
    for rank, r in enumerate(results, start=1):
        r["rank"] = rank
    return results


def format_ranking(ranking, top=8):
    lines = [
        f"{'Rank':<6}{'Tag':<19}{'Score':<9}{'Best-matching text word':<26}Match type",
        "-" * 78,
    ]
    for r in ranking[:top]:
        kind = "identical word" if r["identical"] else "different word"
        lines.append(
            f"{r['rank']:<6}{r['tag']:<19}{r['score']:<9.4f}{r['word']:<26}{kind}"
        )
    return "\n".join(lines)


# ---------------------------------------------------------------------------
# Driver
# ---------------------------------------------------------------------------
def main(text_path=DEFAULT_TEXT):
    print("=" * 78)
    print("Semantic Tagging with GloVe (glove-wiki-gigaword-50) and NumPy")
    print("=" * 78)
```

```
    model = load_model()
    print(f"Model loaded: {len(model.key_to_index)} words, "
          f"dimension {model[next(iter(model.key_to_index))].shape[0]}")

    # Part A / D: tags
    tag_names, T = build_tag_matrix(model, TAGS)
    multi = [t for t in tag_names if len(t.split()) > 1]
    print(f"\n[Part A] {len(tag_names)} tags: {', '.join(tag_names)}")
    print(f"         multi-word tags (mean of component vectors): {multi}")

    # Part B / C: text and preprocessing
    with open(text_path, encoding="utf-8") as f:
        text = f.read()
    raw_count, tokens = preprocess(text)
    in_vocab, oov, W = build_text_matrix(model, tokens)
    distinct_oov = sorted(set(oov))

    print(f"\n[Part B] Input file: {os.path.basename(text_path)}")
    print("\n[Part C] Token statistics")
    print(f"  Tokens before preprocessing        : {raw_count}")
    print(f"  Tokens after preprocessing (m)     : {len(tokens)}")
    print(f"  In-vocabulary tokens (n)           : {len(in_vocab)}")
    print(f"  Out-of-vocabulary tokens           : {len(oov)}")
    print(f"  Distinct out-of-vocabulary tokens  : {len(distinct_oov)}")
    print(f"    {distinct_oov}")
    print(f"  Stopwords added to the starter set : {len(ADDED_STOPWORDS)}")

    # Part D checkpoint
    print("\n[Part D] Checkpoint")
    print(f"  T.shape = {T.shape}")
    print(f"  W.shape = {W.shape}")
    if W.shape[0] == 0:
        sys.exit("ERROR: the input text contains no in-vocabulary words after "
                 "preprocessing (n = 0); the similarity matrix cannot be computed.")
    assert T.shape == (20, 50) and W.shape == (len(in_vocab), 50)

    # Part E / F
    S_vector = np.array(similarity_matrix_vector_class(W, T))
    S_numpy = similarity_matrix_numpy(W, T)
    print("\n[Part E] S_vector.shape =", S_vector.shape, " (Vector class, Python loops)")
    print("[Part F] S_numpy.shape  =", S_numpy.shape, " (W_hat @ T_hat.T)")

    # Part G
    ok, max_diff = verify(S_vector, S_numpy)
    print("\n[Part G] Verification")
    print(f"  np.allclose(S_vector, S_numpy, rtol={RTOL}, atol={ATOL}) -> {ok}")
    print(f"  np.max(np.abs(S_vector - S_numpy)) = {max_diff:.3e}")

    # Part H
    ranking = rank_tags(tag_names, S_numpy, in_vocab)
    print("\n[Part H] Top 8 tags by max-pool score  r_j = max_i S_ij\n")
    print(format_ranking(ranking, top=8))
    ident = [r["tag"] for r in ranking[:8] if r["identical"]]
    diff = [f"{r['tag']} <- {r['word']}" for r in ranking[:8] if not r["identical"]]
    print(f"\n  Top-8 tags matched by an IDENTICAL word : {ident}")
    print(f"  Top-8 tags matched by a DIFFERENT word  : {diff}")

    print("\n  Full ranking of all 20 tags (for reference):\n")
    print(format_ranking(ranking, top=20))
    return ranking


if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else DEFAULT_TEXT)
```

test_semantic_tagging.py

```
"""
Name             : Aman Banka
Registration No. : 261100660030
Course           : AME 5151 - Applied Linear Algebra Lab
Assignment       : Assignment 2 - Semantic Tagging with GloVe and NumPy
Institution      : Manipal School of Information Sciences, MAHE, Manipal
Faculty          : Dr. Devi Prasad
"""

"""
Unit tests for semantic_tagging.py (AME 5151 Assignment 2).

Run with either:
    python -m pytest -v test_semantic_tagging.py
    python -m unittest -v test_semantic_tagging
"""

import unittest

import numpy as np

from vector import Vector
import semantic_tagging as st


class TestSemanticTagging(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        cls.model = st.load_model()
        cls.tag_names, cls.T = st.build_tag_matrix(cls.model, st.TAGS)
        # A small text with punctuation, a hashtag, a number, repetition,
        # stopwords and two made-up OOV words.
        cls.text = ("Our #ResearchExcellence: research at the university, "
                    "research & innovation in 2026! Students build qwzxtech "
                    "and blorptastic labs.")
        _, cls.tokens = st.preprocess(cls.text)
        cls.in_vocab, cls.oov, cls.W = st.build_text_matrix(cls.model, cls.tokens)
        cls.S_vector = np.array(st.similarity_matrix_vector_class(cls.W, cls.T))
        cls.S_numpy = st.similarity_matrix_numpy(cls.W, cls.T)

    # 1. T.shape == (20, 50)
    def test_01_tag_matrix_shape(self):
        self.assertEqual(self.T.shape, (20, 50))
        self.assertEqual(len(self.tag_names), 20)

    # 1b. single-word row equals model[tag]; multi-word row is the mean
    def test_01b_tag_rows(self):
        j = self.tag_names.index("research")
        np.testing.assert_allclose(self.T[j], self.model["research"])
        j = self.tag_names.index("higher education")
        mean = (self.model["higher"].astype(np.float64)
                + self.model["education"].astype(np.float64)) / 2
        np.testing.assert_allclose(self.T[j], mean)

    # 2. W.shape == (n, 50) with n in-vocabulary tokens (repeats kept)
    def test_02_text_matrix_shape(self):
        n = len(self.in_vocab)
        self.assertGreater(n, 0)
        self.assertEqual(self.W.shape, (n, 50))
        self.assertEqual(self.in_vocab.count("research"), 2)   # repetition kept
        for i, tok in enumerate(self.in_vocab):
            np.testing.assert_allclose(self.W[i], self.model[tok])
```

```
    # 3. S.shape == (n, 20)
    def test_03_similarity_shape(self):
        n = len(self.in_vocab)
        self.assertEqual(self.S_numpy.shape, (n, 20))
        self.assertEqual(self.S_vector.shape, (n, 20))

    # 4. one word/tag pair: Vector class agrees with the NumPy entry
    def test_04_single_pair_agrees(self):
        i = self.in_vocab.index("university")
        j = self.tag_names.index("campus")
        expected = Vector(self.model["university"].tolist()).cosine_similarity(
            Vector(self.model["campus"].tolist()))
        self.assertAlmostEqual(self.S_numpy[i, j], expected, places=6)
        # an identical word/tag pair gives exactly cosine 1
        i = self.in_vocab.index("research")
        j = self.tag_names.index("research")
        self.assertAlmostEqual(self.S_numpy[i, j], 1.0, places=12)

    # 5. np.allclose(S_vector, S_numpy) is True
    def test_05_allclose(self):
        self.assertTrue(np.allclose(self.S_vector, self.S_numpy,
                                    rtol=st.RTOL, atol=st.ATOL))
        ok, max_diff = st.verify(self.S_vector, self.S_numpy)
        self.assertTrue(ok)
        self.assertLess(max_diff, 1e-12)

    # 6. OOV token excluded from W and reported as OOV
    def test_06_oov_excluded_and_reported(self):
        for word in ("qwzxtech", "blorptastic", "researchexcellence"):
            self.assertIn(word, self.tokens)
            self.assertIn(word, self.oov)
            self.assertNotIn(word, self.in_vocab)
        self.assertEqual(self.W.shape[0], len(self.tokens) - len(self.oov))

    # 7. a tag with an OOV component raises an error (not silently dropped)
    def test_07_oov_tag_raises(self):
        bad_single = st.TAGS[:19] + ["qwzxtech"]
        with self.assertRaises(ValueError):
            st.build_tag_matrix(self.model, bad_single)
        bad_multi = st.TAGS[:19] + ["research qwzxtech"]
        with self.assertRaises(ValueError):
            st.build_tag_matrix(self.model, bad_multi)

    # extra: preprocessing order and filters
    def test_08_preprocessing(self):
        raw, toks = st.preprocess("The AI-first Lab, at MAHE: we did it!! #Research2026")
        self.assertEqual(raw, 9)
        self.assertEqual(toks, ["first", "lab", "mahe", "research2026"])

    # extra: n = 0 is detected (no usable words)
    def test_09_empty_text_matrix(self):
        _, toks = st.preprocess("we are at the #qwzxtech!")
        in_vocab, oov, W = st.build_text_matrix(self.model, toks)
        self.assertEqual(W.shape, (0, 50))
        self.assertEqual(in_vocab, [])

    # extra: max pooling picks the column maximum and its word
    def test_10_rank_tags(self):
        ranking = st.rank_tags(self.tag_names, self.S_numpy, self.in_vocab)
        self.assertEqual(len(ranking), 20)
        scores = [r["score"] for r in ranking]
        self.assertEqual(scores, sorted(scores, reverse=True))
        for r in ranking:
            j = self.tag_names.index(r["tag"])
            self.assertAlmostEqual(r["score"], self.S_numpy[:, j].max())
```

```
            i = self.in_vocab.index(r["word"])
            self.assertAlmostEqual(self.S_numpy[i, j], r["score"])

    # extra: zero rows are rejected before division
    def test_11_zero_row_rejected(self):
        with self.assertRaises(ValueError):
            st.similarity_matrix_numpy(np.zeros((1, 50)), self.T)


class TestVectorClass(unittest.TestCase):
    def test_vector_ops(self):
        u, v = Vector((3.0, 4.0)), Vector((4.0, 3.0))
        self.assertEqual(u.add(v), Vector((7.0, 7.0)))
        self.assertEqual(u.scale(2), Vector((6.0, 8.0)))
        self.assertEqual(u.dot(v), 24.0)
        self.assertEqual(u.norm(), 5.0)
        self.assertAlmostEqual(u.cosine_similarity(v), 24 / 25)
        with self.assertRaises(ValueError):
            u.cosine_similarity(Vector((0.0, 0.0)))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Appendix D – Input text (`manipal_linkedin_text.txt`)

```
Manipal Academy of Higher Education (MAHE), an Institution of Eminence Deemed to be
University, has achieved a historic milestone by securing the 3rd position in the National
Institutional Ranking

In the Innovation ranking, MAHE proudly stands in the 11-50 band for 2025, while its
research ranking has shown marked improvement, climbing from 23rd in 2024 to an impressive
19th in 2025

As the Lead Support Organisation, Manipal Academy of Higher Education -MAHE was honored to
contribute to a platform that celebrates excellence in healthcare, research, education, and
innovation.

With the World Health Summit (WHS), we asked a bigger question - how can academia move
beyond research to real social impact?

Applications are now open for the Full-Time & Part-Time PhD Program 2025-26 at MAHE (Batch
2)

MCES is thrilled to announce an open call for an Assistant Professor at Manipal Centre for
European Studies.

#MISHA #MAHE #StudentAchievement #ICWA #ResearchExcellence #InternationalRelations

#UndergraduateResearch #ResearchConference #InterdisciplinaryResearch #StudentResearch

Glad to have been part of the Manipal Academy of Higher Education -Microsoft India
collaboration event — a significant step in shaping how we learn, work and innovate in an
AI-first world.

The Department of Student Affairs, MAHE Manipal, successfully conducted the Hostel
Caretakers' Professional Development Program – Series 1

MAHE is all set to embark on Aati da Tulu Parba 2026—a month-long cultural initiative
dedicated to preserving, promoting, and celebrating the rich heritage of Tulunadu.

The Clinical Immersion Program at Manipal welcomes its 18th cohort of engineers from
Becton, Dickinson and Company (BD) on 8–9 October 2026.
```

```
Manipal Academy of Higher Education (MAHE) has signed a Collaboration Agreement with Google
to advance technology-enabled education and strengthen capabilities in Artificial
Intelligence, deep-tech, advanced computing and emerging digital technologies.

Where technology meets healthcare, ideas meet impact, and innovation meets healing.
#SVASTH26 #PSPH #MAHE #Healthcare #HospitalManagement #HealthcareInnovation #DigitalHealth

Dear MITians, Pleased to inform you that Mr. Manu Gupta is illustrious alumnus of Manipal
Institute of Technology (2005, CSE)

The DoC MAHE- Bengaluru recently organized a One-week international Immersion & Educational
Trip to Dubai.

At MAHE- Bengaluru we conducted a bird count last Monday...
```